pfi

Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

Market Calendars and Cleaning

WEEK 1 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note: which days did the market have?

The Desk Lead's Note

Before anyone takes a change, a window or a join on our curve, I need to know which days the market had. Put the weekdays our Treasury file skips in a function, and give Excel the same list. Everyone downloads a holiday calendar: tell me whether it agrees with the file. Check that FRED's dots fall on those same days. Then put series side by side without inventing a day, show me what forward fill does to a change, and find any value that stopped updating. Three tested functions, committed under a message we can search for.

Every cell runs without a key, and the saved outputs are that run. The spreads are synthetic, in bp.

Start the Day

```
work = start_day(3)
Work folder pfi_course3_03_k4w7q2xr: 4 data files, Course 2's bondmath.py, marketdata.py as notebook 02 left it, 2 commits
```

The Tests Pass Before We Start

```
.........   [100%]
9 passed in 0.50s
```

"start_day(3)" copied in the Treasury file, the synthetic spreads, FRED's 10-year sample, and notebook 02's module.

The market's days are the dates the file has: no holiday list decides them.

3.1 Three kinds of day

Notebook 02 left the 10-year with no row on Labor Day. Course 1's Treasury Yields case study counted such weekdays once, with date_range(freq='B') and ".difference". Today the count becomes a calendar.

The Two Files

```
treasury = pd.read_csv('treasury_par_yields.csv', parse_dates=['date'], index_col='date',
                       float_precision='round_trip')
spreads = pd.read_csv('course3_synthetic_spreads.csv', parse_dates=['date'], index_col='date',
                      float_precision='round_trip')
print(len(treasury), 'and', len(spreads), 'rows; the same dates:',
      treasury.index.equals(spreads.index))
print(list(spreads.columns))
2940 and 2940 rows; the same dates: True
['ig_spread_bp', 'hy_spread_bp']
```

Three Counts

```
first, last = treasury.index[0], treasury.index[-1]
# bdate_range is Course 1's date_range(freq='B') as its own function
calendar_days = pd.date_range(first, last)
weekdays = pd.bdate_range(first, last)
market_days = treasury.index
print(f'{first.date()} to {last.date()}: {len(calendar_days):,} calendar days,',
      f'{len(weekdays):,} weekdays, {len(market_days):,} market days')
2015-01-02 to 2026-10-02: 4,292 calendar days, 3,066 weekdays, 2,940 market days
```



A market day is a date the file has.

Excel side by side: three counts

Excel side by side. Dates in A2:A2941; the 10 Yr in B, "ig_spread_bp" in C.

Job	Python	Excel	Result
Calendar days	`len(pd.date_range(first, last))`	`=A2941-A2+1`	4,292
Weekdays	`len(pd.bdate_range(first, last))`	`=NETWORKDAYS(A2,A2941)`	3,066
Market days	`len(treasury.index)`	`=COUNT(A2:A2941)`	2,940

Calendar days, weekdays, market days: every calculation in this course runs on the third.

3.2 The file as its own calendar: missing_weekdays

```
%%writefile -a marketdata.py
def _check_index(index: pd.Index, what: str) -> None:
    """Stop unless `index` is a DatetimeIndex with unique dates in ascending order.
    A helper for the functions below (the leading underscore marks it as not part of the module's API).
    """
    if not isinstance(index, pd.DatetimeIndex):
        raise ValueError(f"{what} must have a DatetimeIndex, not {type(index).__name__}")
    if not index.is_unique:
        raise ValueError(f"{what} has repeated dates, for example {index[index.duplicated()][0].date()}")
    if not index.is_monotonic_increasing:
        raise ValueError(f"{what} dates are not in ascending order")
def missing_weekdays(index: pd.DatetimeIndex) -> pd.DatetimeIndex:
    """The weekdays from the first date to the last that have no row.
    Units: dates. Convention: a weekday is Monday to Friday; no holiday calendar is used, so the result is
    every weekday the data skipped (holidays and any other closure). Weekends are never counted.
    Raises ValueError on an index that is not unique and ascending dates.
    """
    _check_index(index, "index")
    weekdays = pd.bdate_range(index[0], index[-1])
    missing = weekdays.difference(index)
    missing.name = "date"
    return missing
```

Every later notebook asks which weekdays the file skips, so it becomes a function.

Read both docstrings first. "_check_index" stops on an index that is not unique dates in ascending order; its leading underscore marks a helper inside the module. "pd.Index" and "-> None" read as notebook 02's "str | None" did: what goes in, and what comes back, here nothing.

126, as Dates

```
importlib.reload(md)
missing = md.missing_weekdays(treasury.index)
print(len(missing), 'weekdays with no row, from', missing[0].date(), 'to', missing[-1].date())
126 weekdays with no row, from 2015-01-19 to 2026-09-07
```

Excel side by side. "=NETWORKDAYS(A2,A2941)-COUNT(A2:A2941)" returns 126.

The seven of 2026, and the first test

The Seven of 2026

```
missing_2026 = missing[missing.year == 2026]
for day in missing_2026:
    print(day.date(), day.strftime('%A'))
print(len(treasury.loc['2026']), 'rows in 2026')
2026-01-01 Thursday
2026-01-19 Monday
2026-02-16 Monday
2026-05-25 Monday
2026-06-19 Friday
2026-07-03 Friday
2026-09-07 Monday
190 rows in 2026
```

The First Test

```
def test_missing_weekdays_hand_index():
    # Friday 2026-09-04 to Wednesday 2026-09-09, with Monday (Labor Day) and Tuesday absent
    index = pd.DatetimeIndex(["2026-09-04", "2026-09-09"])
    missing = md.missing_weekdays(index)
    # the weekend is never counted; the two weekdays are
    assert list(missing) == [pd.Timestamp("2026-09-07"), pd.Timestamp("2026-09-08")]
.........   [100%]
10 passed in 0.50s
```

"missing_weekdays" makes the file its own calendar: 126 dates, and as Excel's holiday list the seven of 2026 give the file's 190 rows.

Excel side by side. H2:H8 hold the seven dates: Excel's holiday list.

Job	Python	Excel	Result
Day of the week, Monday first	`pd.Timestamp('2026-09-07').dayofweek: 0`	`=WEEKDAY(DATE(2026,9,7),2)`	1
Day of the week, Excel's default		`=WEEKDAY(DATE(2026,9,7))`	2: Sunday is 1
Weekdays of 2026, to 2 October	`len(pd.bdate_range('2026-01-01', last))`	`=NETWORKDAYS(DATE(2026,1,1),DATE(2026,10,2))`	197
Less the list	`len(treasury.loc['2026'])`	`=NETWORKDAYS(DATE(2026,1,1),DATE(2026,10,2),H2:H8)`	190
September, less the list	`len(treasury.loc['2026-09'])`	`=NETWORKDAYS(DATE(2026,9,1),DATE(2026,9,30),H2:H8)`	21

pandas counts Monday as 0, Excel's "WEEKDAY(...,2)" as 1.

3.3 A holiday calendar is not the market calendar

Tutorials and AI answers reach for pandas' federal holiday calendar.

pandas' Federal Holidays

```
from pandas.tseries.holiday import GoodFriday, USFederalHolidayCalendar
# return_name=True gives a Series: each holiday's name, indexed by its date
federal = USFederalHolidayCalendar().holidays(first, last, return_name=True)
print(len(federal), 'federal holidays, all on weekdays:', (federal.index.dayofweek < 5).all())
121 federal holidays, all on weekdays: True
```

pandas moves a Saturday holiday to the Friday before and a Sunday one to the Monday after.

The course gives no reason for any of these days: the file is the record.



3.4 FRED's dots are the same days

Notebook 02 drew two ways of saying nothing: FRED's dot and the file's absent row, on Labor Day.

The Sample's Dot

```
payload = json.loads(Path('course3_fred_format_dgs10.json').read_text(encoding='utf-8'))
sample = md.parse_fred_observations(payload, 'DGS10')
dots = sample.index[sample.isna()]
print('Dots:', list(dots.strftime('%Y-%m-%d')),
      '| each a missing weekday:', dots.isin(missing).all())
Dots: ['2026-09-07'] | each a missing weekday: True
```

The Numbers Fall on the File's Rows

```
file_span = treasury.loc[sample.index[0]:sample.index[-1], '10 Yr']
sample_numbers = sample.dropna()
print('Same dates:', sample_numbers.index.equals(file_span.index),
      '| largest gap:', (sample_numbers - file_span).abs().max(), 'percent')
Same dates: True | largest gap: 0.0 percent
```

FRED Since 2015

With your key, the live cell asks FRED for the 10-year since 2015 and checks every dot.

This product uses the FRED® API but is not endorsed or certified by the Federal Reserve Bank of St. Louis.

The Key Cell

```
try:
    fred_key = md.get_secret("FRED_API_KEY", env_file=ENV_FILE)
    print("FRED key: found. It is held in memory and is never printed.")
except RuntimeError as error:
    fred_key = None
    print(f"FRED key: not found. Live cells will be skipped. {error}")
FRED key: not found.
Live cells will be skipped.
Secret FRED_API_KEY not found.
Looked in: the environment variable FRED_API_KEY, Colab's Secrets panel, and the file .env given as env_file.
```

Every Dot Since 2015

```
if fred_key is None:
    print(NO_KEY_MESSAGE)
else:
    live_dgs10 = md.fred_series('DGS10', fred_key, start='2015-01-01')
    live_end = min(live_dgs10.index[-1], last)
    live_span = live_dgs10.loc[first:live_end]
    live_dots = live_span.index[live_span.isna()]
    live_expected = missing[missing <= live_end]
    assert list(live_dots) == list(live_expected), 'a dot that is not a missing weekday'
    assert (live_span.index.dayofweek < 5).all(), 'a weekend row'
    live_numbers = live_span.dropna()
    live_gap = (live_numbers - treasury.loc[:live_end, '10 Yr']).abs().max()
    assert live_numbers.index.equals(treasury.loc[:live_end].index) and live_gap <= 1e-9
    print(f'{len(live_span):,} rows, {len(live_dots)} dots, each a missing weekday;',
          f'largest gap {live_gap:.1e} percent')
Live cell skipped: no FRED key was found. Every other cell runs without one.
```

With your key, it prints FRED's rows since 2015, its dots, every one a weekday the file skips, and a largest gap of 0.

3.5 Joining two series: align

Course 1's Combining and Loading drew what "merge" keeps with "how=". "align" joins dated series the same way, on their dates, inner or outer.

Append align

```
def align(series: dict[str, pd.Series], how: str = "inner") -> pd.DataFrame:
    # stop on a join the course does not use
    if how not in ("inner", "outer"):
        raise ValueError(f"how must be 'inner' or 'outer', not {how!r}")
    for name, values in series.items():
        _check_index(values.index, name)
    # one column per series, named by its key
    frame = pd.concat(series, axis=1, join=how)
    # an outer join can mix the order of dates, so sort them
    frame = frame.sort_index()
    frame.index.name = "date"
    return frame
```

"pd.concat" with "axis=1", as in Course 1's Combining and Loading, here takes a dict, "dict[str, pd.Series]", whose keys name the columns. "join=" picks inner or outer.

Where Outer Earns Its Place

```
four_month = treasury['4 Mo'].dropna()
inner = md.align({'4 Mo': four_month, 'ig_spread_bp': spreads['ig_spread_bp']})
outer = md.align({'4 Mo': four_month, 'ig_spread_bp': spreads['ig_spread_bp']}, how='outer')
print(len(inner), 'rows inner;', len(outer), 'outer, with', outer['4 Mo'].isna().sum(), 'NaN')
989 rows inner; 2940 outer, with 1951 NaN
```

Inner drops almost eight years of spreads; outer keeps them and leaves the bill's cells empty, as they were.

Two tests, and Excel side by side

Two Tests

```
def test_align_inner_keeps_common_dates():
    dates = pd.to_datetime(["2026-09-03", "2026-09-04", "2026-09-08"])
    ten = pd.Series([5.20, 5.22, 5.25], index=dates)
    spread = pd.Series([98.0, 98.4], index=dates[[0, 2]])
    inner = md.align({"10 Yr": ten, "ig_spread_bp": spread})
    assert list(inner.index) == list(dates[[0, 2]])
    assert list(inner.columns) == ["10 Yr", "ig_spread_bp"]
    # an outer join keeps every date and leaves the gap empty, never filled
    outer = md.align({"10 Yr": ten, "ig_spread_bp": spread}, how="outer")
    assert len(outer) == 3 and math.isnan(outer.loc["2026-09-04", "ig_spread_bp"])
def test_align_rejects_unknown_how():
    ten = pd.Series([5.20], index=pd.to_datetime(["2026-09-03"]))
    with pytest.raises(ValueError):
        md.align({"10 Yr": ten}, how="left")
```

Excel side by side.

Job	Python	Excel	Result
A date with no row		`=XLOOKUP(DATE(2026,9,7),A2:A2941,B2:B2941)`	#N/A
The next market day	`both.loc['2026-09-08', 'file']`	`=XLOOKUP(DATE(2026,9,8),A2:A2941,B2:B2941)`	4.8

3.6 Forward fill, for display only

".ffill()" fills forward: each "NaN" takes the last value above it.

Excel side by side. T2:T4 hold Friday's 4.78, an empty Labor Day, and Tuesday's 4.8; U2 is "=T2".

Job	Excel	Result
Fill forward	`=IF(T3="",U2,T3)`	4.78
Labor Day's change	`=(U3-U2)*100`	0
Tuesday's change	`=(U4-U3)*100`	1.9999999999999574, as Python's before .round(2)

Over the Whole Sample

```
for name, changes in [('market rows', change_rows), ('filled', change_filled)]:
    print(f'{name}: {changes.count()} changes, {(changes == 0).sum()} of them 0,',
          f'standard deviation {changes.std():.3f} bp')
market rows: 41 changes, 3 of them 0, standard deviation 4.909 bp
filled: 42 changes, 4 of them 0, standard deviation 4.854 bp
```

One filled day adds a zero and pulls the standard deviation down.

Forward fill invents a zero change on a day with no market: fill a displayed copy only, never before a change, a window, or a join that feeds a number.

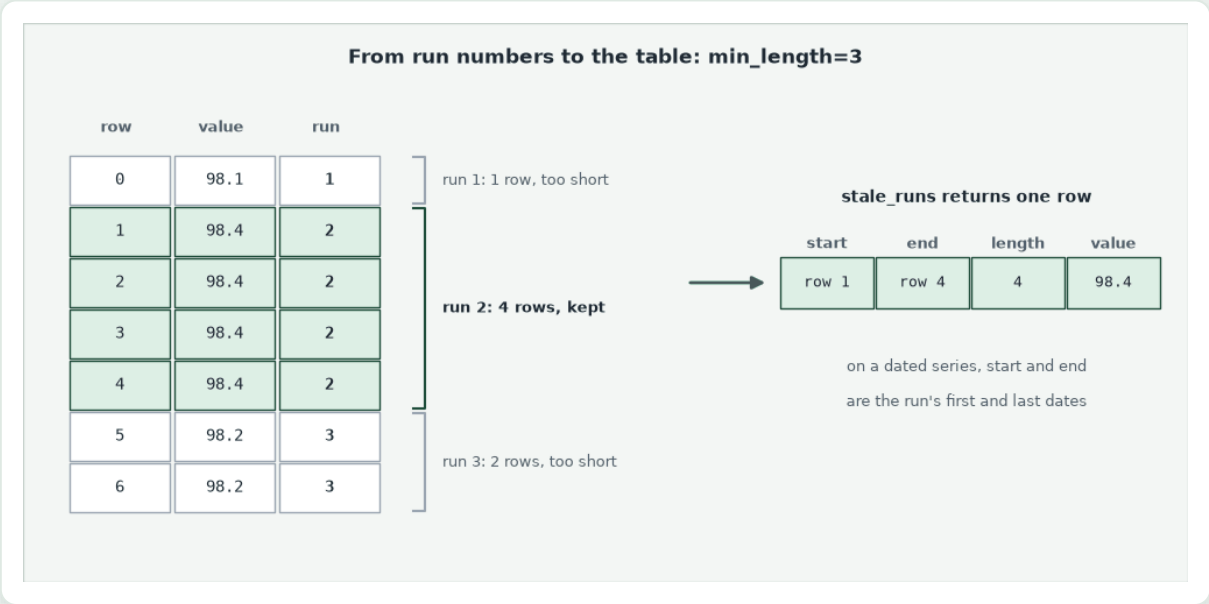
3.7 Stale values: a run, counted by hand

A gap is a day with no value. A stale value is a row that is there and stopped moving: a feed that froze, a quote nobody updated.

".shift()" moves each value down one row, and the first row gets "NaN".

```
hand = pd.Series([98.1, 98.4, 98.4, 98.4, 98.4, 98.2, 98.2])
steps = pd.DataFrame({'value': hand, 'before': hand.shift()})
steps['differs'] = steps['value'] != steps['before']
steps['run'] = steps['differs'].cumsum()
steps
```

	value	before	differs	run
0	98.1	NaN	True	1
1	98.4	98.1	True	2
2	98.4	98.4	False	2
3	98.4	98.4	False	2
4	98.4	98.4	False	2
5	98.2	98.4	True	3
6	98.2	98.2	False	3



A new run starts wherever a value differs from the one before.

Append stale_runs, docstring first

```
%%writefile -a marketdata.py
def stale_runs(series: pd.Series, min_length: int = 3) -> pd.DataFrame:
    """Runs of the same value repeated on consecutive observations: a sign of a value that stopped updating.
    Units: `value` has the series' units, `length` counts observations. Convention: consecutive means
    consecutive rows, whatever the calendar gap. One row per run of at least `min_length` identical values,
    with columns start, end, length, value.
    Raises ValueError if min_length is below 2, the series has missing values (drop them first), or its index
    is not unique ascending dates.
    """
    _check_index(series.index, "series")
    if min_length < 2:
        raise ValueError(f"min_length must be at least 2, not {min_length}")
    if series.isna().any():
        raise ValueError(f"series has {series.isna().sum()} missing values; drop them first with .dropna()")
    run_id = (series != series.shift()).cumsum()
    runs = []
    for _, run in series.groupby(run_id):
        if len(run) >= min_length:
            runs.append({"start": run.index[0], "end": run.index[-1], "length": len(run), "value": run.iloc[0]})
    return pd.DataFrame(runs, columns=["start", "end", "length", "value"])
```

It refuses "NaN": a gap is not a run, so drop it first.

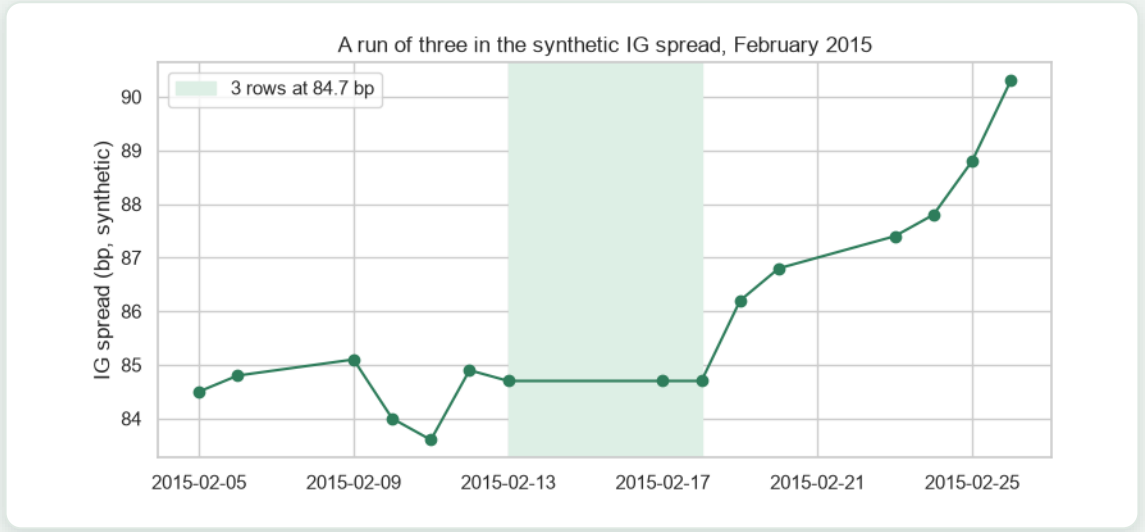
"for _, run in series.groupby(run_id)" visits each run in turn; "_" sets the run number aside, as in Course 2.

"pd.DataFrame(runs, columns=...)" names the columns even when no run is found.

Runs of three in the synthetic spreads

```
importlib.reload(md)
ig, hy = spreads['ig_spread_bp'], spreads['hy_spread_bp']
ig_runs = md.stale_runs(ig)
print(len(ig_runs), 'IG runs of 3 or more;', len(md.stale_runs(hy)), 'HY')
ig_runs
4 IG runs of 3 or more; 0 HY
```

	start	end	length	value
0	2015-02-13	2015-02-18	3	84.7
1	2021-12-08	2021-12-10	3	81.3
2	2024-02-14	2024-02-16	3	107.2
3	2025-11-21	2025-11-25	3	92.2



Three rows over six calendar days: Monday 16 February had no row, and a run counts rows.

Excel side by side. D4 holds "=AND(C4=C3,C3=C2)", filled down to D2941: TRUE where a spread equals the two rows above it.

Job	Python	Excel	Result
Runs of 3 or more	`len(md.stale_runs(ig))`	`=COUNTIF(D4:D2941,TRUE)`	4
Where	`ig_runs['end']`	the rows showing TRUE	2015-02-18, 2021-12-10, 2024-02-16, 2025-11-25

Longer, shorter, and a real yield

```
for length in [5, 2]:
    print(f'runs of {length} or more: IG {len(md.stale_runs(ig, length))},',
          f'HY {len(md.stale_runs(hy, length))}')
two_year_runs = md.stale_runs(treasury['2 Yr'])
print(len(two_year_runs), '2 Yr runs of 3 or more; the longest:')
two_year_runs[two_year_runs['length'] == two_year_runs['length'].max()]
runs of 5 or more: IG 0, HY 0
runs of 2 or more: IG 107, HY 29
84 2 Yr runs of 3 or more; the longest:
```

	start	end	length	value
54	2020-12-16	2020-12-28	8	0.13
65	2021-05-10	2021-05-19	8	0.16

"stale_runs" works on any series without gaps, "True" and "False" included: there it reports runs of each, and "value" says which.

The Last Test

Its dates are a hand series: any seven weekdays do.

```
def test_stale_runs_finds_a_run():
    dates = pd.bdate_range("2026-09-01", periods=7)
    values = pd.Series([98.1, 98.4, 98.4, 98.4, 98.4, 98.2, 98.2], index=dates)
    runs = md.stale_runs(values, min_length=3)
    # one run of four, from the second row to the fifth; the pair at the end is too short
    assert len(runs) == 1
    assert runs.loc[0, "start"] == dates[1] and runs.loc[0, "end"] == dates[4]
    assert runs.loc[0, "length"] == 4 and runs.loc[0, "value"] == 98.4
.............   [100%]
13 passed in 0.50s
```

A run is a reason to look, not a verdict: no run of five or more in the spreads, and a two-decimal yield sat still for eight market days.

3.8 Fix the message before anything builds on it

The note's last ask: a commit under a message we can search for.

```
!git -C "{work}" status --short
!git -C "{work}" diff --stat
 M marketdata.py
 M test_marketdata.py
 marketdata.py    | 73 ++++++
```

Bring it home: into my_bondmath

Copy the work folder's two files into "my_bondmath", test them, and commit. The copy replaces your two files, so anything of your own must be in the work folder first. In a PowerShell terminal in VS Code, with the course's ".venv" active:

```
$work = "$env:TEMP\pfi_course3_03_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\marketdata.py", "$work\test_marketdata.py" .
python -m pytest -q test_marketdata.py
git status --short
git diff --stat
git add marketdata.py test_marketdata.py
git commit -m "Notebook 03: missing_weekdays, align, and stale_runs"
```

pytest shows "13 passed", and the DGS10 sample the tests read is there since notebook 02. A slip in your own message is fixed with "git commit --amend -m" while the commit has not left your machine. In Colab, download the two files from the work folder under "/tmp" instead.

You Can Now

Count calendar days, weekdays and market days, and say which one a calculation runs on.

Find the weekdays a file skips with "missing_weekdays", and give Excel the same list.

Show where pandas' federal holiday calendar disagrees with the file, both ways.

Check that FRED's dots fall on the file's missing weekdays.

Join series on their dates with "align", inner or outer, filling nothing.

Say what forward fill does to a change, and keep it for display.

Find values that stopped moving with "stale_runs", and fix a commit message with "--amend".

Predict first: WORKDAY and the federal calendar

Predict 1

Excel's "=WORKDAY(DATE(2026,9,4),1)" asks for the working day after Friday 4 September, with no holiday list. It returns:

A) Monday 2026-09-07

B) Tuesday 2026-09-08

C) Saturday 2026-09-05

```
friday = pd.Timestamp('2026-09-04')
print('Next weekday:   ', weekdays[weekdays > friday][0].date())
print('Next market day:', market_days[market_days > friday][0].date())
```

Predict 2

pandas lists 121 holidays; the file skips 126 weekdays. How many of pandas' holidays have a row in the file anyway? Write down a number.

```
shared = missing.intersection(federal.index)
not_federal = missing.difference(federal.index)
federal_with_rows = federal.index.difference(missing)
print(len(shared), 'in both,', len(not_federal), 'skipped but not federal,',
      len(federal_with_rows), 'federal with a row')
federal[federal_with_rows]
```

Predict first: the outer join

Predict 3

Join the file's 10 Yr with the parsed sample, which still holds its dot as "NaN", outer. Labor Day:

A) is left out, since a "NaN" row is no date

B) is a row, "NaN" in both columns

C) is a row with Friday's value carried in

```
importlib.reload(md)
both = md.align({'file': file_span, 'fred': sample}, how='outer')
print(len(both), 'rows')
both.loc['2026-09-03':'2026-09-09']
```

The AI check

The docstring is the prompt. The code that comes back runs, and has one data bug.

1

Docstring

Units, conventions, and what it must never do: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: closed_days's test, written from its docstring, fails: AssertionError: a weekday with no row is missing.

Code that runs is not the same as code that is right.

The note, answered

The note asked	Answer
The weekdays the file skips	missing_weekdays: 126, and as Excel's list the seven of 2026 give its 190 rows
FRED's dots	The same days, tied on the sample and live
Side by side, no day invented	align, inner or outer, nothing filled
Forward fill	A zero change on Labor Day: display only
Values that stopped	stale_runs: 4 IG runs of three, none of five or more
Three functions, committed	13 passed, the message amended

Be proud: from today every series in this course runs on the market's own days.

Next: Resampling and the As-Of Value (notebook 04) asks which of these days stands for a week or a month.